Sorbonne Université
Master Informatique – Spécialité SESI

Rapport de Projet – PSESI

Projet SESI : Conception de circuits intégrés à l'aide d'outils open source

Auteur :
Ousmane DIALLO

Encadrant :
Dimitry GALAYKO

Mai 2026





Table des matières

1 Introduction **1**
1.1 Contexte 1
1.2 Objectif du projet 1

2 Le circuit Bandgap **1**
2.1 Principe du circuit 1
2.2 Problématique du startup 3
2.3 Dimensionnement 3

3 Flot de Conception **3**
3.1 Xschem - saisie du schéma 4
3.1.1 Schéma : 4
3.1.2 Simulation : Integration avec Ngspice 4
3.1.3 Génération de la netlist pour klayout : 6
3.2 Klayout - saisie du layout 6
3.2.1 Cas du ptap : 6
3.2.2 Cas des résistances : 6
3.2.3 Vérifications du layout 6
3.3 Magic - Extraction parasitaire 8
3.3.1 Séquence d'extraction 8
3.3.2 Problème rencontré avec l'extraction de pnpMPA : 9
3.4 Xschem - Simulation Post Layout 9

4 Comparaison avec Cadence **10**

5 Conclusion **11**

1 Introduction

1.1 Contexte

Historiquement, la conception de circuits intégrés analogiques est faite sur des outils sous licence tels que Cadence. Ces outils sont utilisés dans l'industrie, et nous apprenons à les prendre en main lors de nos TP.
Cependant ces outils ont quelques défauts :

- Licences coûteuses, limitant son utilisation aux structures ayant un certain budget.
- Opacité des technologies de fabrication : elles sont verrouillées par des accords de non divulgation, limitant ainsi le partage des travaux.

Des technologies open-source existent, telle que IHP-Open-PDK, proposée par le Leibniz Institute for High Performance Microelectronics. Elle permet au grand public de concevoir des circuits réels, pouvant par la suite être fabriqués en fonderie.
L'utilisation d'outils open source a plusieurs avantages :

- Maîtrise totale de l'environnement de travail : il peut être installé sur n'importe quelle machine compatible, de plus il n'y a pas besoin de licences.
- Partage plus simple du travail : tout étant open source, les travaux de recherche peuvent être partagés librement sur des plateformes telles que GitHub. Ils peuvent donc être consultés, vérifiés et reproduits par d'autres chercheurs, sans qu'ils n'aient à avoir de licence ou à signer de NDA.

1.2 Objectif du projet

Actuellement les travaux pratiques des enseignements en master/licence et les projets de recherche au LIP6 sont faits sous Cadence, un logiciel sous licence. Ce logiciel est largement utilisé dans l'industrie. La maîtrise de ce logiciel est utile pour les étudiants souhaitant travailler en entreprise, mais comme vu précédemment, des outils opensource pourraient être attractifs pour le monde de la recherche.
Contrairement à Cadence qui est présent et éprouvé depuis les années 90, l'utilisation d'outils opensource ne s'est démocratisée que récemment, avec la sortie de technologies libres à partir de 2020, avec notamment les technologies SkyWater 130nm (2020) et GlobalFoundries 180nm (2022). La technologie que nous allons utiliser dans le cadre de ce projet est la PDK IHP SG13G2, sortie en 2023, contrairement aux deux autres qui se limitent à des circuits CMOS, elle permet de concevoir des circuits BiCMOS. Elle ouvre donc plus de possibilité.
Cependant, vu que ces technologies sont assez récentes, nous ne savons pas si l'environnement de développement autour est assez complet pour être utilisé dans le cadre Universitaire. C'est pour cette raison que nous allons commencer par prendre ces outils en main sur des circuits basiques, puis concevoir un circuit un peu plus exigeant, afin de voir s'ils peuvent être utilisés lors de projets plus concrets.
Le but du projet est donc de maîtriser tout le flot de conception d'un circuit intégré analogique, de la saisie du schéma, jusqu'au layout en passant par les différentes étapes de vérification.

2 Le circuit Bandgap

2.1 Principe du circuit

Dans le cadre de ce projet, je vais réaliser un circuit Bandgap : il s'agit d'un générateur de tension de référence indépendant de la tension d'alimentation et de la température ambiante. Le circuit fonctionne en mettant en série deux tensions : une qui augmente quand la température augmente (PTAT) et une qui baisse quand la température augmente (CTAT). Il suffit de dimensionner le circuit de façon à ce que ces deux comportements se compensent, puis les additionner afin de générer une tension indépendante de la température. Grâce à la polarisation du circuit, on obtient

une tension de sortie indépendante de la tension d'alimentation (tant qu'elle est assez élevée pour placer tous les transistors dans leur bon mode de fonctionnement).

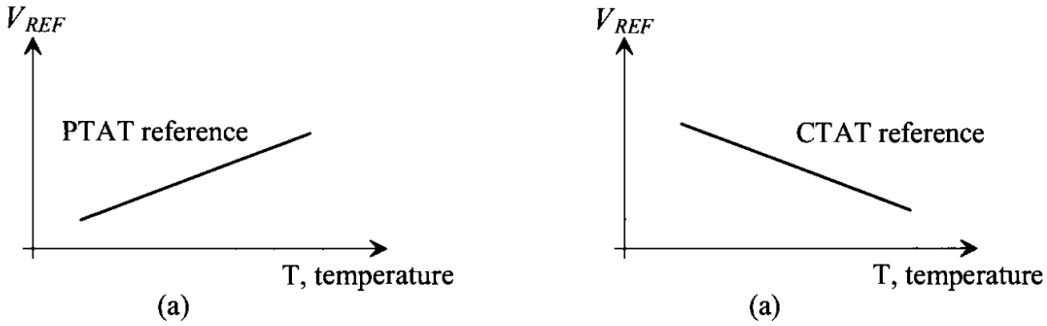


Figure 23.1 (a) PTAT and (b) CTAT voltage references.

FIGURE 1 – Références PTAT et CTAT [1, p. 745]

Le circuit se décompose en 4 blocs :

1. Les dipôles non linéaires (Rouge) :
   Constitués de deux transistors bipolaires montés en diode (Q1 et Q2, de tailles différentes) et d'une résistance R1. Ils exploitent la relation tension-courant des jonctions PN pour créer un point de croisement unique. Ce bloc sert de capteur thermique fondamental pour générer le courant de référence.
2. Le court circuit actif (Bleu) :
   Réalisé par quatre transistors MOS (un miroir de courant pMOS et une paire nMOS), ce bloc impose simultanément l'égalité des courants et des tensions aux bornes des dipôles non linéaires, forçant le circuit à se stabiliser sur son point de fonctionnement.
3. La tension PTAT (Vert) :
   Acronyme de Proportional To Absolute Temperature. Le miroir de courant (M1.2 M1.3) recopie le courant produit par les dipôles linéaires et le court circuit actif. Ce courant est converti en tension à l'aide de la résistance R2. Ainsi, la tension PTAT se retrouve aux bornes de R2. Cette tension augmente linéairement avec la température.
4. La tension CTAT (Orange) :
   Acronyme de Complementary To Absolute Temperature. La tension de jonction brute Vbe du transistor bipolaire possède un coefficient thermique négatif naturel. Cette tension diminue linéairement avec la température.

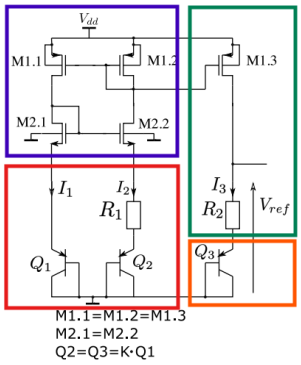


FIGURE 2 – Générateur de tension stable "bandgap" [2]

2.2 Problématique du startup

Le cœur du Bandgap étant un système auto-polarisé basé sur une boucle de contre-réaction positive, il possède intrinsèquement deux points d'équilibre stables :

— Le point nominal recherché (où le circuit génère la bonne tension de référence).
— Un point à l'origine (0, 0), où tous les courants et tensions du circuit sont nuls.

Si le circuit se trouve à ce point mort lors de la mise sous tension, il n'en sortira jamais spontanément car aucun courant ne circule pour amorcer la boucle.
Pour résoudre ce problème, un circuit de démarrage (Startup) est indispensable [3]. Dans notre architecture, il prend la forme d'un transistor auxiliaire (XMStartup) connecté de manière à injecter un courant initial dans la branche de polarisation lors de la mise sous tension. Cette perturbation force le circuit à s'écarter définitivement de son point d'équilibre nul pour basculer vers son point de fonctionnement nominal.
Une fois que la boucle est active et que les courants nominaux s'établissent, le potentiel des nœuds internes remonte, ce qui vient bloquer automatiquement le transistor de démarrage. Ainsi, après cette phase d'amorçage, le bloc de Startup reste complètement éteint et ne perturbe pas la stabilité ou la consommation du Bandgap en régime permanent.

2.3 Dimensionnement

Pour réaliser le dimensionnement de la cellule Bandgap dans cette technologie, j'ai suivi le TP de l'UE ECFA : « *CADENCE - TP2 : Conception et simulation d'un bandgap* » de Dimitri Galayko. [2] Il était initialement prévu pour la technologie *ams C35* en 350 nm.
Pour trouver les valeurs adaptées des composants, j'ai réalisé des simulations paramétriques afin de tester différentes valeurs et de sélectionner celles qui sont les plus adaptées dans chaque cas. La procédure pour réaliser des simulations paramétriques sera expliquée dans la partie du flot de conception.
Dans ce projet, je ne me suis pas focalisé sur la précision du circuit réalisé, mais plutôt sur la prise en main du flot de conception.
Voici les paramètres de la `FIGURE 2` retenus pour le circuit :
$L = 5\mu m, W = 2\mu m, K = 1, R1 = 56K, R2 = 389K$
Avec W et L les dimensions des transistors MOS

Note pour les résistances :
Pour pouvoir réaliser le layout, il faudra utiliser les résistances de la technologie. Pour les dimensionner on utilise directement leurs tailles (w et l). La technologie permet aussi de définir des résistances en serpentin qui prendront une surface plus compacte sur le layout.
Cependant, le calcul de la valeur de la résistance en serpentin n'est pas homogène entre les différents logiciels du flot de conception, et les paramètres qui la définissent ne sont pas actuellement pris en compte dans le logiciel Magic, que nous utiliserons pour l'extraction parasitaire. [1] A cette étape de la conception, il ne faut introduire aucun repliement (bend) dans les résistances. L'optimisation dans l'espace se fera au niveau du layout.

3 Flot de Conception

Cette section détaille l'environnement logiciel mis en œuvre pour réaliser le circuit, depuis la saisie du schéma initial jusqu'aux simulations post-layout.
La conception d'un circuit intégré analogique repose sur un enchaînement d'étapes (le design flow), où chaque logiciel remplit une fonction bien spécifique. Contrairement à Cadence qui propose un environnement entièrement unifié, le choix a été fait ici d'utiliser un flot composé de plusieurs outils open-source indépendants :

1. Discussion sur le sujet : `https://github.com/IHP-GmbH/IHP-Open-PDK/issues/746`

— Xschem pour la saisie du schéma électrique (schematic) et le pilotage des simulations.
— Ngspice comme moteur de simulation analogique pour valider le comportement électrique du circuit.
— Klayout pour le dessin des masques physiques (layout) et les vérifications géométriques et topologiques (DRC et LVS).
— Magic pour l'extraction des éléments parasites (PEX), permettant d'effectuer par la suite des simulations post-layout.

3.1 Xschem - saisie du schéma

3.1.1 Schéma :

La saisie de schéma sur Xschem se fait de la même manière que sur Cadence. On place des instances de cellules, puis on les connecte grâce à des fils. On définit les entrées et les sorties de notre schéma grâce à des pins.

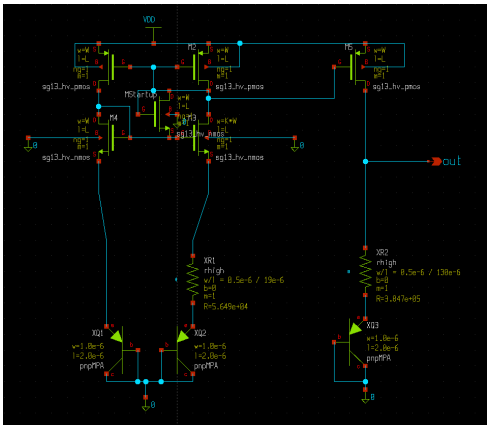


FIGURE 3 – Schéma Bandgap saisi sur Xschem

3.1.2 Simulation : Integration avec Ngspice

La particularité de Xschem est que la simulation se fait directement dans l'interface du schéma. On peut donc créer un fichier testbench.sch dans lequel on place la cellule à tester, le code pour la simulation spice ainsi qu'un graphe pour visualiser les résultats. Contrairement aux outils cadence, il n'y a pas d'interface dédiée pour configurer et lancer des simulations.

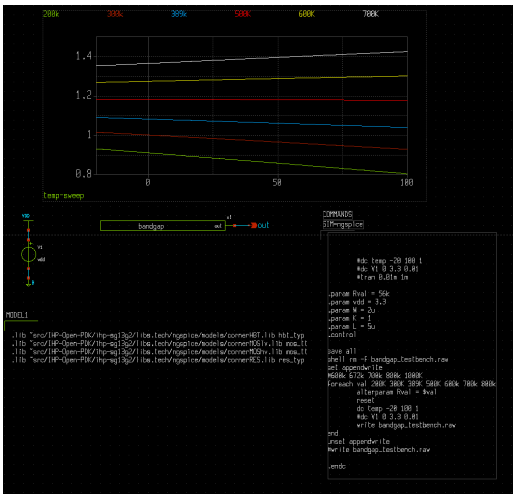


FIGURE 4 – Exemple de schéma avec simulation

Exemple : simulation paramétrique pour différentes valeurs de résistances
On utilise le symbole `simulator_commands_shown.sym` dans lequel on écrit le code de simulation spice

Listing 1 – Code pour une simulation paramétrique de Rval en température

```
name=COMMANDS
simulator=ngspice
only_toplevel=false
value="
.param Rval = 56k
.param vdd = 3.3
.param W = 2u
.param K = 1
.param L = 5u

.control
save all
shell rm -f bandgap_testbench.raw
set appendwrite
foreach val 389K 600k 672k 700k 800k 1000K
    alterparam Rval = $val
    reset
    dc temp -20 100 1
    #dc V1 0 3.3 0.01 # mise en commentaire pour selectionner la
       simulation que l'on veut faire
    write bandgap_testbench.raw
end
unset appendwrite
.endc
"
```

Ici, on a une simulation paramétrique nécessitant l'utilisation d'une boucle `foreach`, le code est plus simple pour les simulations classiques.

La librairie de Xschem et celle de IHP propose des exemples de simulations. Il est possible de s'en inspirer. Lancement de la simulation :

1. Génération de la netlist [Netlist]
2. Simulation [Simulate]
3. Importation des résultats [Waves] » [Load first analysis found], puis sélectionner le fichier .raw dans lequel on a écrit les résultats de la simulation.

Pour visualiser les résultats, on utilise les graphes dans [Simulation] » [Graphs] » [Add waveform graph]
On ouvre le bloc en double cliquant. On peut plot les signaux du circuit en sélectionnant directement dans la liste de gauche. Afin d'afficher notre simulation paramétrique, on va rédiger de la façon suivante : `Nom_de_la_courbe ; signal % numéro_du_paramètre`

Listing 2 – Signaux à plot dans le graphdialog

```
200k ; out%0
300k ; out%1
389k ; out%2
500K ; out%3
600K ; out%4
700K ; out%5
```

On obtient ce graphique où l'on peut distinguer les courbes :

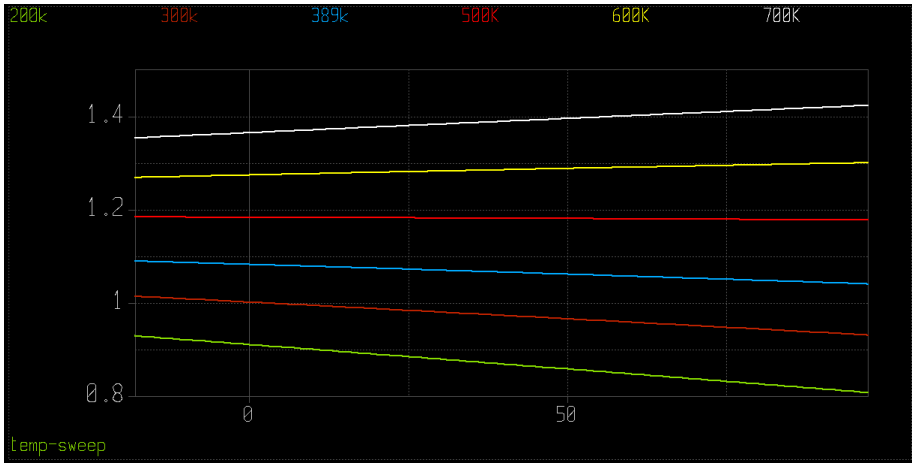


FIGURE 5 – Plot de la simulation paramétrique de Rval en température

3.1.3 Génération de la netlist pour klayout :

Sélectionner dans les menus : Simulation » LVS » LVS netlist + Top level is a .subckt Xschem génère une netlist adaptée au LVS de klayout, il suffira de faire correspondre le nom du subckt et le nom de la cellule TOP.

3.2 Klayout - saisie du layout

Dans cette étape, nous dessinons les différents masques pour la création de la puce.
On commence par placer les différentes cellules de notre netlist en renseignant les paramètres. Ensuite on connecte les cellules à l'aide de path sur les différentes couches de métal.

3.2.1 Cas du ptap :

Afin de polariser les caissons, on utilise des ntaps et ptaps. Des cellules existent pour ces composants, cependant, si elles ne sont pas présentes sur la netlist, Klayout générera une erreur lors des vérifications. Pour régler ce problème, j'ai dessiné mes propres ptaps et ntaps en respectant la succession de couches demandées par la techno. Il devrait aussi être possible de placer ces cellules initialement dans le schéma Xschem et ainsi utiliser les cellules dédiées sur Klayout.

3.2.2 Cas des résistances :

Pour créer une résistance dans cette technologie on utilise des cellules (rhigh) dont on va adapter la longueur pour avoir une résistance plus ou moins grande. Comme vu précédemment, pour la rendre plus compacte, on peut introduire des repliements pour mettre la résistance sous forme de serpentin. Cependant, on ne va pas directement utiliser le paramètre de la cellule `rhigh`. Pour obtenir une taille plus compacte, on va diviser la résistance en segments en gardant la même longueur totale. L'outil de LVS est configuré pour détecter ces résistances en série, et les fusionner ensemble. Ainsi on se retrouve avec une résistance qui est aux bonnes dimensions.

3.2.3 Vérifications du layout

La vérification du layout se fait en deux étapes :

— Le Design Rule Checking (DRC) : Le logiciel vérifie si le dessin du layout respecte les règles imposées par la techno. Ces règles sont décrites dans un document pdf consultable par l'utilisateur dans le repertoire `/IHP-Open-PDK/ihp-sg13g2/libs.doc/doc` .

Lancement du DRC : SG13G2 PDK > Run Klayout DRC
On peut visualiser les résultats du DRC dans Tools > Marker Browser . Les erreurs de DRC sont listées, en cliquant dessus, le logiciel recadre la fenêtre vers là où se trouve l'erreur et nous indique l'intitulé de la règle qui n'est pas respectée. Si le nom de la cellule est en vert, on n'a pas d'erreur de DRC.

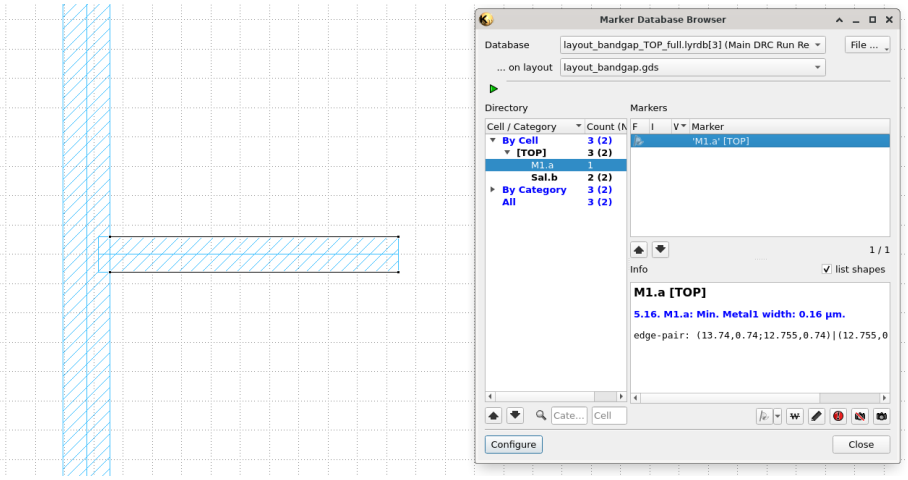


FIGURE 6 – Fenêtre DRC : Règle `5.16. M1.a: Min. Metal1 width: 0.16 µm.` non respectée. Il faut tracer une piste plus large pour cette couche

— Le Layout Versus Schematics : Le logiciel vérifie si le layout saisi correspond bien à la netlist du schéma. Pour qu'il puisse comparer on lui fournit la netlist dans SG13G2 PDK > SG13G2 LVS Options dans le champ Netlist Path : .
Lancement du LVS : SG13G2 PDK > Run Klayout LVS
On peut visualiser les résultats du LVS dans Tools > Netlist Browser. L'onglet Cross Reference permet de comparer le layout avec la netlist fournie.
Exemple d'erreur LVS : Ici la résistance n'a pas le même paramètre `l` sur le layout et sur la netlist.

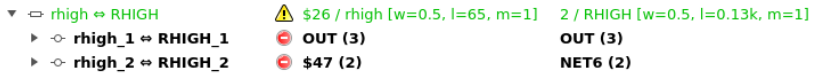


FIGURE 7 – Klayout : Erreur LVS sur une rhigh

Le LVS est vérifié une fois que tous les éléments sont au vert.

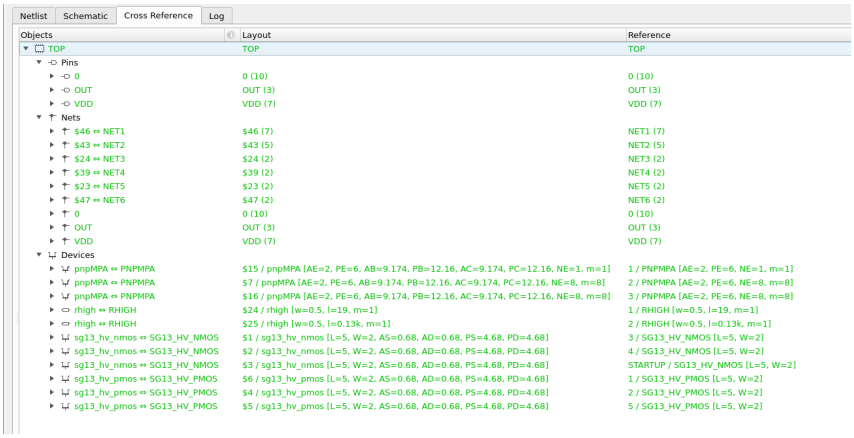


FIGURE 8 – LVS sans aucune erreur

Une fois ces deux tests passés, il reste une étape pour savoir si notre layout est exploitable : l'extraction des parasites.

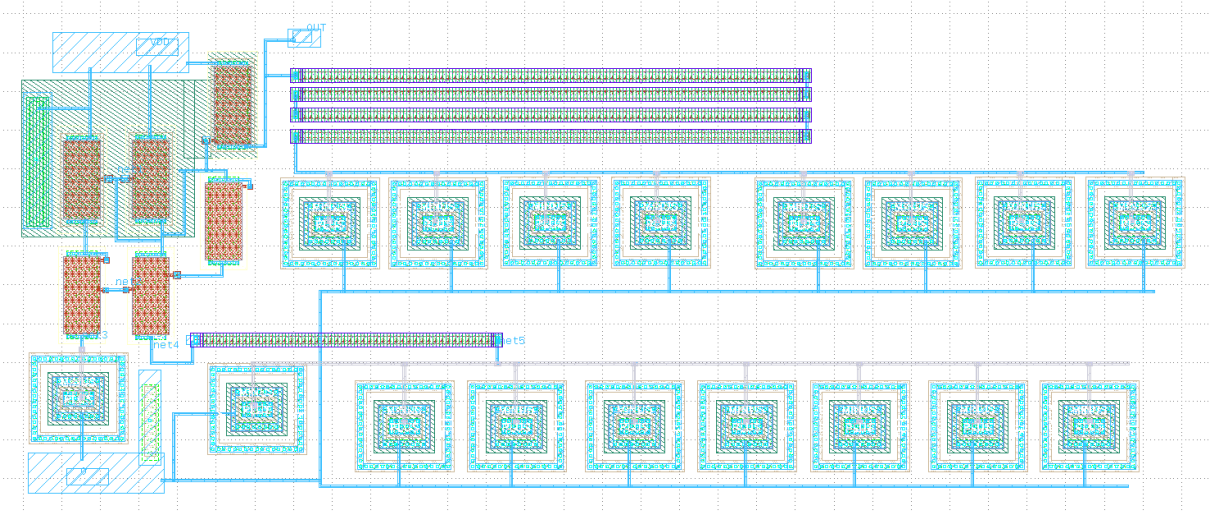


FIGURE 9 – Layout du bandgap sur Klayout

3.3 Magic - Extraction parasitaire

Klayout, que nous avons utilisé pour sa praticité et qui a été conseillé par des professionnels du domaine, ne possède pas d'outils d'extractions de parasites.
Nous devons donc ajouter un nouveau logiciel dans notre flot de conception : Magic.
Magic permet d'extraire à partir du layout la netlist comprenant les différents parasites produit par les différentes couches. Il calcule les valeurs des résistances et capacités parasites en fonction de la géométrie du layout. On pourra par la suite la simuler sur Xschem afin de voir si le fonctionnement du composant avec ces parasites respecte toujours le cahier des charges. Les règles et modèles mathématiques nécessaires à cette extraction sont fournis par la technologie (IHP) et centralisés dans le fichier de configuration technologique (.tech) situé dans le répertoire `IHP-Open-PDK/ihp-sg13g2/libs.tech/magic` du PDK.

3.3.1 Séquence d'extraction

Pour réaliser l'extraction des éléments parasites (PEX) sous Magic, la séquence de commandes suivante doit être exécutée dans la console du logiciel :

Listing 3 – Séquence de commandes pour l'extraction parasite sous Magic

```
gds read patch/file.gds
load TOP
select top cell
extract all
ext2spice format ngspice
ext2spice cthresh 0 rthresh 0
ext2spice
```

On récupère ensuite le fichier TOP.spice contenant la netlist extraite dans le répertoire d'où a été lancé Klayout.

3.3.2 Problème rencontré avec l'extraction de pnpMPA :

Lors de l'étape d'extraction sous l'outil Magic (v8.3.643) avec l'Open-PDK SG13G2 d'IHP (v0.5.0 beta), les transistors bipolaires pnpMPA n'étaient pas présents dans la netlist générée. J'ai donc isolé le problème en créant un fichier .gds contenant seulement 1 transistor pnpMPA. La netlist extraite était vide dans ce cas.
Une erreur possible a été trouvée dans les fichiers tech du PDK. Pour distinguer les grands caissons d'isolation de puissance des petits caissons de composants de signal, le PDK utilise des opérations morphologiques de rabotage (shrink) et de dilatation (grow) sur le calque DNWELL. Le caisson DNWELL du transistor pnpMPA était donc caractérisé comme `large_dnwell`. Or la règle d'extraction du transistor pnpMPA exige la présence d'un caisson dnwell de taille standard. L'analyse du fichier `ihp-sg13g2-cifin.tech` montre que le seuil de 2.28µm est insuffisant pour le DNWELL de la PCell officielle (3.4×2.88µm). Le fix proposé (seuil à 1500nm) a été documenté dans une issue sur le GitHub de la distribution du PDK. [2]
Pour forcer l'extraction du pnpMPA, j'ai passé la valeur de seuil de 1140 à 1500. Ainsi, le pnpMPA n'est plus extrait en tant que `large_dnwell` et apparaît bien dans la netlist extraite.

3.4 Xschem - Simulation Post Layout

La première étape est de créer un symbole pour notre cellule extraite.

- Modifier le nom du sous-circuit principal (généralement nommé .subckt TOP par défaut dans le fichier texte) pour lui donner le nom exact de la cellule, à savoir (exemple : .subckt nom_layout).
- Créer un fichier .sym et changer son paramètre (touche Q) en "type=primitive" pour pouvoir la lier à une netlist spice et non un schéma.
- ajouter un `code_shown.sym` avec `''.include path/fichier_layout.spice"`
- inclure une instance du .sym dans le schéma testbench.
- lancer la simulation comme vu précédemment

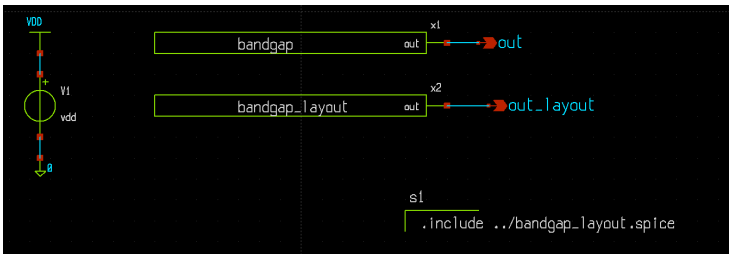


FIGURE 10 – Schéma du testbench schematic vs layout

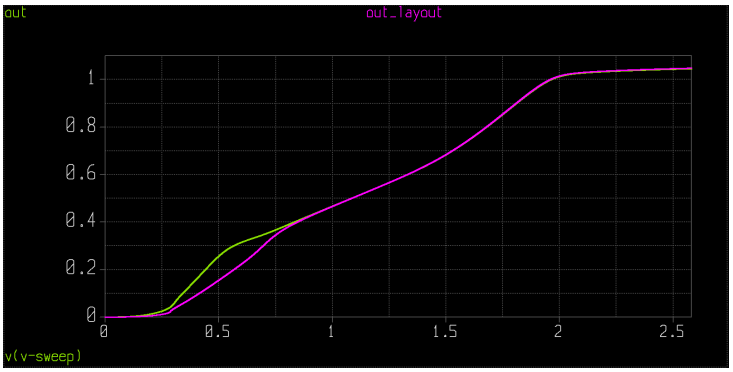


FIGURE 11 – Comparaison bandgap schéma vs layout sur une analyse DC

2. Lien vers l'issue : `https://github.com/IHP-GmbH/IHP-Open-PDK/issues/974`

4 Comparaison avec Cadence

L'objectif est ici de comparer Xschem et Cadence sur le plan de la simulation. Puisque les deux outils exploitent la technologie SG13G2 d'IHP, on peut s'attendre à obtenir les mêmes résultats en sortie. Pour débuter cette étude, la saisie du schéma a d'abord été réalisée sous Cadence.

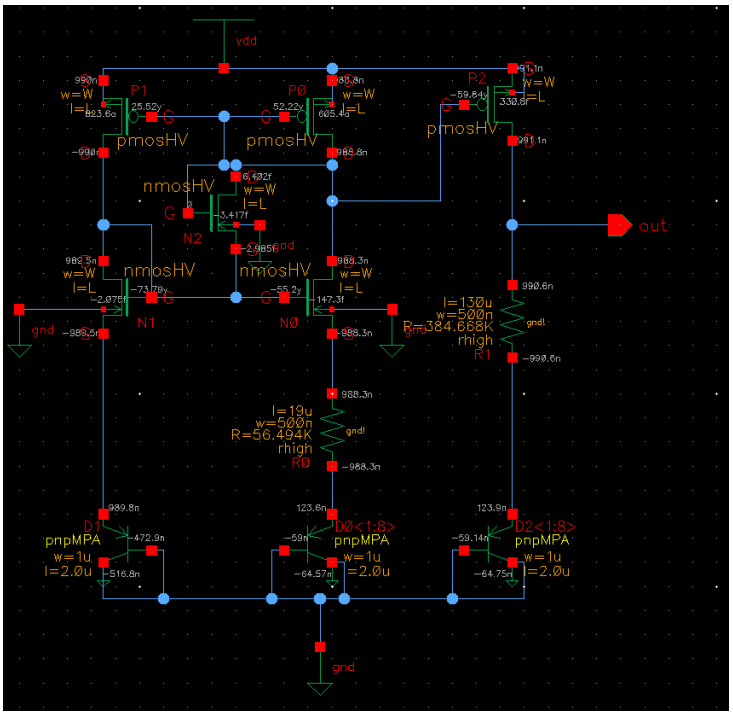


FIGURE 12 – Schéma du bandgap saisi sur Cadence

Après simulation, en superposant les deux courbes on s'aperçoit qu'elles ne sont pas exactement les mêmes, surtout à des tensions d'alimentation inférieures à 1V. Il y a donc des différences dans les modèles de calculs des deux versions de la technologie.

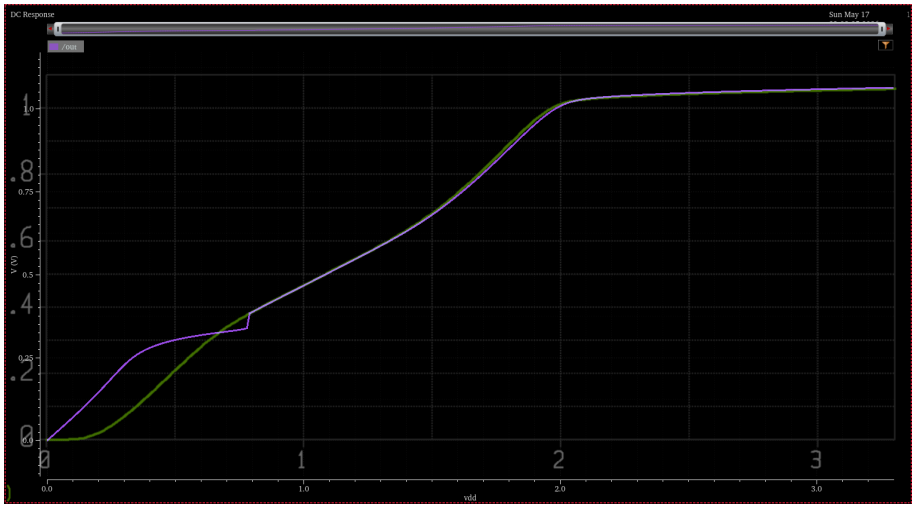


FIGURE 13 – Comparaison simulation DC Xschem (Vert) et Cadence(Violet)

5 Conclusion

Ce projet m'a permis de découvrir concrètement l'architecture d'un circuit Bandgap tout en testant un flot de conception analogique entièrement open-source. La réalisation de ce travail met en évidence plusieurs points clés :

- L'écosystème est encore en développement : les logiciels utilisés évoluent constamment et des problèmes sont corrigés au fur et à mesure.
- Une prise en main plus complexe : Maîtriser ce flot demande un réel effort de transition quand on a l'habitude des outils industriels traditionnels tels que Cadence. La communauté reste très active autour de ces logiciels libres en proposant une documentation riche et de nombreux guides pratiques en ligne.
- L'absence d'automatisation : contrairement à Cadence où l'on peut créer des layouts automatiquement, les outils open-source actuels ne permettent que de designer des layouts entièrement à la main. Cela reste viable pour des petits circuits, mais peut devenir ennuyant pour de plus gros projets.

Malgré ces quelques points de friction, l'adoption et le retour d'expérience sur ces outils libres restent le meilleur moyen de soutenir leur maturité et de participer activement à leur démocratisation dans le monde académique et de la recherche.

Références

[1] R. Jacob Baker. *CMOS : Circuit Design, Layout, and Simulation.* Wiley-IEEE Press, 3rd edition, 2010.

[2] Dimitry Galayko. Cadence - tp2 : Conception et simulation d'un bandgap.

[3] Behzad Razavi. *Design of Analog CMOS Integrated Circuits.* McGraw-Hill, international edition, 2001.

Dans le cadre de ce projet, afin d'approfondir la prise en main de ce flot de conception open-source et de documenter la méthodologie appliquée, un ensemble de ressources d'ingénierie, de documentations techniques, de dépôts officiels ainsi que des supports pédagogiques audiovisuel ont été consultés, exploités ou développés

Dépôt du Projet (Ousmane Diallo) *Code source et livrables* — Dépôt GitHub personnel centralisant l'intégralité des fichiers de conception, incluant les schémas électriques Xschem, les masques physiques KLayout, ainsi que les netlists extraites associées aux simulations post-layout.
`https://github.com/dlo-ousmane/psesi`

Chaîne de M. Maślanka *Tutoriels Vidéos Xschem & Magic* — Série de guides pratiques détaillant l'interopérabilité entre la saisie de schéma sous Xschem et l'extraction parasitaire (PEX) sous Magic.
`https://www.youtube.com/@marcinmaslanka9621/videos`

Chaîne du QNFCF *Tutoriels Vidéos KLayout* — Formations vidéo et démonstrations pas-à-pas pour la prise en main de l'éditeur de masques KLayout et l'analyse des fenêtres de rapport DRC/LVS.
`https://www.youtube.com/@qnfcf6093/videos`

IHP-Open-PDK *SG13G2 (130nm BiCMOS)*, IHP Microelectronics. Dépôt GitHub officiel (mars 2023). `https://github.com/IHP-GmbH/IHP-Open-PDK`

Documentation Oceane (ivref) *Générateur de tension de référence* — Manuel technique officiel hébergé sur les serveurs du LIP6, détaillant l'application et les spécifications d'Oceane pour les circuits de référence.
`https://largo.lip6.fr/sesi-docs/oceane-docs/documents/ivref/ivref.pdf`

Xschem *Schematic capture and netlisting EDA tool*, S. Schippers. Documentation officielle du projet (2016). `https://xschem.sourceforge.io/stefan/index.html`

KLayout *High performance layout viewer and editor*, M. Köfferlein. Documentation officielle du projet (2006). `https://www.klayout.de/`

Ngspice *Open Source Mixed Mode / Mixed Level Circuit Simulator*, Équipe de développement Ngspice. Documentation officielle. `https://ngspice.sourceforge.io/`

Magic *VLSI Layout Tool & Parasitic Extractor*, Open Circuit Design. Documentation officielle. `http://opencircuitdesign.com/magic/`

Cadence Virtuoso *Suite de conception industrielle (Référence commerciale)*, Cadence Design Systems. `https://www.cadence.com/`